

Le QCM Python & pandas

20 questions notées /20



Avant ton entretien Data Analyst. Nettoyage, groupby, merge, typage, NaN. Durée conseillée : 25 min.

Objectif : mesurer ton pandas réel, pas seulement ta syntaxe. Ce test vérifie si tu sais quand un `astype(int)` plante sur des NaN, quand un `groupby` avale silencieusement des lignes et pourquoi un `merge` peut doubler un total sans lever d'erreur.

Repères : 12/20 = niveau junior correct, 17/20 = profil qu'on garde.

Candidat _____ Date _____ Note _____ /20

Observation terrain Datafuji : sur les tests pandas que je corrige, ce ne sont jamais les grandes questions d'algorithmie qui font perdre des points. C'est un `astype(int)` qui plante sur une colonne pleine de NaN, un `groupby` qui avale des lignes sans le dire, ou un `merge` qui double un total sans erreur. Le code tourne, le chiffre est faux, et personne ne s'en aperçoit avant le debrief.

Palier 1 : le fichier brut qu'on te confie.

Quatre questions sur le nettoyage : doublons, noms de colonnes sales, renommage, filtre texte.

Question 1. Pour retirer les doublons d'un DataFrame en gardant la première occurrence sur la colonne email, la bonne méthode est :

- a) `df['email'].unique()`, qui ne renvoie qu'un tableau de valeurs, pas un DataFrame filtré.
- b) `df.drop_duplicates(subset=['email'], keep='first')`
- c) `df.drop_duplicates(subset=['email'], keep='last')`, qui garde la dernière occurrence, pas la première.
- d) `df.groupby('email').first()`, qui change la structure du DataFrame.

Question 2. Tu reçois un CSV où les noms de colonnes contiennent des espaces en trop, par exemple ' age '. La correction la plus fiable est :

- a) `df.columns = df.columns.str.strip()`
- b) `df.rename(columns={'age': 'age'})`, qui ne change rien tant que le nom exact avec espaces n'est pas connu.
- c) `df.columns = df.columns.str.lower()`, qui traite la casse, pas les espaces.
- d) `df.columns.str.strip()` sans réassignation, qui ne modifie rien en place.

Question 3. Un collègue écrit `df.rename(columns={'nom': 'name'})` puis utilise `df['name']` et obtient une `KeyError`. L'erreur vient de :

- a) `rename()` ne peut pas renommer une seule colonne à la fois.
- b) La colonne nom n'existait pas dans le DataFrame d'origine.
- c) `rename()` renvoie une copie par défaut; sans réassigner ou passer `inplace=True`, `df` n'est pas modifié.
- d) `columns` doit être un tuple, pas un dictionnaire.

Question 4. Pour garder les lignes où la colonne ville contient la sous-chaîne 'saint', sachant que la colonne a des valeurs manquantes, la meilleure option est :

- a) `df[df['ville'] == 'saint']`, une égalité stricte.
- b) `df[df['ville'].str.contains('saint')]`, qui casse ou produit des NaN dans le masque à cause des valeurs manquantes.
- c) `df[df['ville'].isin(['saint'])]`, même limite que l'égalité stricte.
- d) `df[df['ville'].str.contains('saint', na=False)]`

Palier 2 : sortir des chiffres qui tiennent.

Quatre questions sur groupby et l'agrégation, là où un chiffre juste en apparence peut être faux.

Question 5. Que renvoie `df.groupby('categorie').agg({'vente': 'sum'})` ?

- a) Un DataFrame indexé par categorie, avec une colonne vente contenant la somme par groupe.
- b) Une Series indexée par vente.
- c) Le DataFrame original avec une colonne supplémentaire.
- d) Une erreur, `agg()` attend une liste de fonctions.

Question 6. Tu veux la vente moyenne par categorie, sous forme de Series indexée par categorie. Tu écris :

- a) `df['vente'].groupby().mean()`
- b) `df.groupby('categorie').mean()`, qui moyenne toutes les colonnes numériques, pas seulement vente.
- c) `df.groupby('categorie')['vente'].mean()`
- d) `df.mean().groupby('categorie')`

Question 7. Un collègue enchaîne `df.groupby(['annee', 'mois']).sum()` puis essaie `resultat['annee']` et obtient une `KeyError`. Le problème :

- a) `groupby()` avec deux clés renvoie toujours une erreur.
- b) Le résultat garde `annee` et `mois` comme MultiIndex, pas comme colonnes; il manque `reset_index()` ou `as_index=False`.
- c) `sum()` ne fonctionne pas avec un `groupby` à deux clés.
- d) Il fallait utiliser `pivot_table` à la place.

Question 8. Ta colonne de regroupement contient des valeurs manquantes. Que fait pandas par défaut avec `df.groupby('cle').sum()` ?

- a) Il regroupe toutes les valeurs manquantes ensemble dans un groupe NaN.
- b) Il lève une erreur, `groupby` refuse les clés NaN.
- c) Il remplace les NaN par 0 avant de grouper.
- d) Il exclut les lignes dont la clé de regroupement est NaN, sans avertissement.

Palier 3 : recoller les tables.

Quatre questions sur `merge` : jointure par défaut, doublons de clés, noms différents, collisions de colonnes.

Question 9. Sans préciser `how`, `pd.merge(df1, df2, on='id')` effectue :

- a) Une jointure interne (`how='inner'` par défaut).
- b) Une jointure gauche.
- c) Une jointure externe complète.
- d) Une concaténation verticale.

Question 10. Après `pd.merge(commandes, clients, on='client_id')`, le nombre de lignes du résultat est bien supérieur à celui de commandes. La cause la plus probable :

- a) `merge()` a un bug connu qui duplique aléatoirement des lignes.
- b) La colonne `client_id` contient des valeurs dupliquées des deux côtés, ce qui produit un produit cartésien partiel sur ces clés.
- c) Il manquait `how='inner'`.
- d) `clients` contient plus de lignes que `commandes`.

Question 11. Les tables ont des noms de clé différents : `id_client` dans `commandes`, `client_id` dans `clients`. Pour les joindre, on écrit :

- a) `pd.merge(commandes, clients, on='id_client')`
- b) `pd.merge(commandes, clients, on=['id_client','client_id'])`
- c) `pd.concat([commandes, clients], axis=1)`
- d) `pd.merge(commandes, clients, left_on='id_client', right_on='client_id')`

Question 12. Les deux DataFrames contiennent tous deux une colonne `date`. Après `merge`, tu obtiens `date_x` et `date_y` sans t'y attendre. La cause :

- a) `merge()` renomme aléatoirement les colonnes en cas de conflit.
- b) Il fallait utiliser `join()` à la place de `merge()`.
- c) pandas ajoute automatiquement les suffixes `_x/_y` sur les colonnes en collision; passe `suffixes=(...)` pour choisir des noms explicites.
- d) La colonne `date` a été convertie en index par erreur.

Palier 4 : les pièges de type.

Quatre questions sur les dtypes : conversion, colonnes mixtes, coercion, mémoire.

Question 13. Ta colonne `age` est de type `float` et contient des `NaN`. Tu appelles `df['age'].astype(int)`. Que se passe-t-il ?

- a) Les `NaN` sont automatiquement convertis en 0.
- b) La colonne reste inchangée, `astype()` est ignoré silencieusement.
- c) Les `NaN` sont convertis en le plus petit entier possible.
- d) pandas lève une erreur : impossible de convertir un `NaN` vers un type entier natif; il faudrait passer par le type nullable `Int64` ou combler les `NaN` avant conversion.

Question 14. Une colonne d'un CSV contient à la fois des nombres et du texte ('12' et 'inconnu'). Après `pd.read_csv`, le dtype de cette colonne est :

- a) `object`, pandas ne peut pas choisir un type numérique unique et retombe sur le type générique.
- b) `float64`, avec 'inconnu' converti en `NaN` automatiquement.
- c) `int64`, les valeurs texte étant ignorées.
- d) `category`, pandas détecte l'hétérogénéité et optimise le stockage.

Question 15. Que fait `pd.to_numeric(df['prix'], errors='coerce')` ?

- a) Il lève une exception dès qu'une valeur n'est pas convertible.
- b) Il convertit les valeurs numériques et remplace celles non convertibles par `NaN`.
- c) Il ignore silencieusement la colonne si une seule valeur pose problème.
- d) Il force toutes les valeurs en chaîne de caractères.

Question 16. La colonne `statut` ne contient que 4 valeurs possibles répétées sur un million de lignes. Pour réduire l'empreinte mémoire, la meilleure option est :

- a) La convertir en `float64`.
- b) La laisser en `object`, pandas optimise déjà les chaînes répétées.
- c) `df['statut'] = df['statut'].astype('category')`
- d) La convertir en index du DataFrame.

Palier 5 : les trous qui faussent tout.

Quatre questions sur les valeurs manquantes : les trois marqueurs, la décision métier, le comptage, le piège de comparaison.

Question 17. Quelle affirmation décrit correctement les trois marqueurs de valeur manquante en pandas ?

- a) NaN, None et pd.NA sont strictement interchangeables dans tous les cas.
- b) NaN est un float, None est l'objet Python générique (souvent dans les colonnes object), et pd.NA est le marqueur des types nullable introduits depuis pandas 1.0 (comme Int64 ou boolean).
- c) pd.NA remplace totalement NaN depuis pandas 1.0, NaN n'existe plus.
- d) None ne peut apparaître que dans une Series de type float64.

Question 18. Un rapport mensuel contient des lignes avec des valeurs manquantes sur la colonne montant. Faut-il les combler (fillna) ou les supprimer (dropna) ?

- a) Toujours `fillna(0)`, pour ne jamais perdre de lignes.
- b) Toujours `dropna()`, les valeurs manquantes faussent systématiquement une moyenne.
- c) Le choix est indifférent, pandas gère les deux de la même façon en interne.
- d) Ça dépend de l'impact métier : si l'absence cache une vraie donnée manquante, `dropna()` évite de fausser les totaux ; si elle représente une absence connue (pas de vente ce jour), `fillna(0)` est plus fidèle.

Question 19. Que renvoie `df.isna().sum()` ?

- a) Une Series indiquant, pour chaque colonne, le nombre de valeurs manquantes.
- b) Le nombre total de valeurs manquantes du DataFrame, sous forme d'un entier unique.
- c) Un DataFrame booléen de même forme que df.
- d) Une erreur, `sum()` ne s'applique pas sur un DataFrame booléen.

Question 20. Un collègue filtre les valeurs manquantes avec `df[df['montant'] == np.nan]` et obtient toujours un DataFrame vide, même quand il y a des NaN. L'erreur :

- a) Il fallait écrire `np.NaN` avec une majuscule.
- b) `montant` doit être converti en float avant la comparaison.
- c) La comparaison `== np.nan` renvoie toujours False, quelle que soit la valeur ; il faut utiliser `df['montant'].isna()`.
- d) `np.nan` n'existe que dans NumPy, pas en pandas.

Corrigé, chaque piège nommé.

Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute.

1. **b)** `drop_duplicates` retire les doublons et garde la ligne choisie; `unique()` ne renvoie qu'un tableau de valeurs, pas un `DataFrame`.
2. **a)** `str.strip()` nettoie les espaces sur l'ensemble des noms de colonnes en une ligne, contrairement à `lower()` ou à `rename()` sans connaître le nom exact.
3. **c)** `rename()` sans `inplace=True` renvoie une copie; sans réaffectation, le `DataFrame` d'origine ne change pas.
4. **d)** `na=False` force les valeurs manquantes à `False` dans le masque au lieu de faire planter le filtre.
5. **a)** `agg()` avec un dictionnaire produit un `DataFrame` indexé par la clé de `groupby`, avec une colonne par agrégation demandée.
6. **c)** `groupby(...)[ 'vente' ].mean()` sélectionne la colonne avant d'agréger; sans le crochet, toutes les colonnes numériques sont moyennées.
7. **b)** Avec deux clés de `groupby`, le résultat a un `MultiIndex`; `reset_index()` ou `as_index=False` sont nécessaires pour retrouver des colonnes.
8. **d)** C'est le piège classique : les lignes à clé `NaN` disparaissent silencieusement du résultat, sans erreur ni avertissement.
9. **a)** `inner` est la valeur par défaut de `how` dans `pd.merge`; seules les clés présentes des deux côtés survivent.
10. **b)** Une clé dupliquée des deux côtés du `merge` multiplie les lignes correspondantes : c'est un produit cartésien partiel, pas un bug.
11. **d)** `left_on/right_on` permettent de joindre sur deux colonnes de noms différents, contrairement à `on` qui exige un nom commun.
12. **c)** Les suffixes `_x/_y` sont ajoutés automatiquement sur toute colonne en collision; `suffixes=(...)` permet de les nommer explicitement.
13. **d)** Un `float NaN` ne peut pas être casté vers un entier natif (`int64`) : `pandas` lève une erreur. Le type nullable `Int64` tolère les `NaN`, pas `int` natif.
14. **a)** Dès qu'une colonne mélange types numérique et texte, `pandas` ne peut pas choisir un `dtype` numérique et retombe sur `object`.
15. **b)** `errors='coerce'` est fait pour ça : toute valeur non convertible devient `NaN` au lieu de faire planter la conversion.
16. **c)** `astype('category')` stocke les valeurs répétées comme des codes entiers plutôt que des chaînes dupliquées, ce qui réduit fortement la mémoire.
17. **b)** C'est la distinction à connaître : `NaN` est un `float IEEE`, `None` est l'objet Python générique, `pd.NA` est le marqueur unifié des types nullable apparu en `pandas 1.0`.
18. **d)** Il n'y a pas de règle universelle `fillna/dropna` : la bonne réponse dépend de ce que représente l'absence de donnée pour le métier.
19. **a)** `isna()` produit un masque booléen de même forme que `df`; `sum()` agrège ensuite colonne par colonne, donnant un compte par colonne.
20. **c)** `np.nan` n'est jamais égal à lui-même : `== np.nan` renvoie toujours `False`. `isna()` est la seule façon fiable de détecter les valeurs manquantes.

Grille de lecture du score

0 à 8/20 : reprends les bases de nettoyage (Palier 1) et de `groupby` (Palier 2) avant tout entretien.

9 à 12/20 : niveau junior correct, mais les pièges `merge` et `typage` (Paliers 3-4) te feront perdre des points en live coding.

13 à 16/20 : bon niveau, revois surtout le palier `NaN` (Palier 5), c'est souvent le dernier détail qui fait la différence en debrief.

17 à 20/20 : profil qu'on garde. Tu sais expliquer chaque piège à voix haute, pas seulement écrire le code qui marche.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis (SQL, dbt, git, entretien technique), dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

